4 Caching

Forget About It

> *In the practical use of our intellect, forgetting is as important a function as remembering.*
>
> —WILLIAM JAMES

You have a problem. Your closet is overflowing, spilling shoes, shirts, and underwear onto the floor. You think, "It's time to get organized." Now you have two problems.

Specifically, you first need to decide what to keep, and second, how to arrange it. Fortunately, there is a small industry of people who think about these twin problems for a living, and they are more than happy to offer their advice.

On what to keep, Martha Stewart says to ask yourself a few questions: "How long have I had it? Does it still function? Is it a duplicate of something I already own? When was the last time I wore it or used it?" On how to organize what you keep, she recommends "grouping like things together," and her fellow experts agree. Francine Jay, in *The Joy of Less,* stipulates, "Hang all your skirts together, pants together, dresses together, and coats together." Andrew Mellen, who bills himself as "The Most Organized Man in America," dictates, "Items will be sorted by type—all slacks together, shirts together, coats, etc. Within each type, they're further sorted by color and style—long-sleeved or short-sleeved, by neckline, etc." Other than the sorting problem this could entail, it looks like good advice; it certainly seems unanimous.

Except that there is another, larger industry of professionals who also think obsessively about storage—and they have their own ideas.

Your closet presents much the same challenge that a computer faces when managing its memory: space is limited, and the goal is to save both money and time. For as long as there have been computers, computer scientists have grappled with the dual problems of what to keep and how to arrange it. The results of these decades of effort reveal that in her four-sentence advice about what to toss, Martha Stewart actually makes several different, and not fully compatible, recommendations—one of which is much more critical than the others.

The computer science of memory management also reveals exactly how your closet (and your office) ought to be arranged. At first glance, computers appear to follow Martha Stewart's maxim of "grouping like things together." Operating systems encourage us to put our files into folders, like with like, forming hierarchies that branch as their contents become ever more specific. But just as the tidiness of a scholar's desk may hide the messiness of their mind, so does the apparent tidiness of a computer's file system obscure the highly engineered chaos of how data is actually being stored underneath the nested-folder veneer.

What's really happening is called *caching*.

Caching plays a critical role in the architecture of memory, and it underlies everything from the layout of processor chips at the millimeter scale to the geography of the global Internet. It offers a new perspective on all the various storage systems and memory banks of human life—not only our machines, but also our closets, our offices, our libraries. And our heads.

The Memory Hierarchy

> *A certain woman had a very sharp consciousness but almost no memory….She remembered enough to work, and she worked hard.*
>
> —LYDIA DAVIS

Starting roughly around 2008, anyone in the market for a new computer has encountered a particular conundrum when choosing their storage option. They must make a tradeoff between *size* and *speed*. The computer industry is currently in transition from hard disk drives to solid-state drives; at the same price point, a hard disk will offer dramatically greater

capacity, but a solid-state drive will offer dramatically better performance—as most consumers now know, or soon discover when they begin to shop.

What casual consumers may not know is that this exact tradeoff is being made within the machine itself at a number of scales—to the point where it's considered one of the fundamental principles of computing.

In 1946, Arthur Burks, Herman Goldstine, and John von Neumann, working at the Institute for Advanced Study in Princeton, laid out a design proposal for what they called an electrical "memory organ." In an ideal world, they wrote, the machine would of course have limitless quantities of lightning-fast storage, but in practice this wasn't possible. (It still isn't.) Instead, the trio proposed what they believed to be the next best thing: "a hierarchy of memories, each of which has greater capacity than the preceding but which is less quickly accessible." By having effectively a pyramid of different forms of memory—a small, fast memory *and* a large, slow one—maybe we could somehow get the best of both.

The basic idea behind a memory hierarchy should be intuitive to anyone who has ever used a library. If you are researching a topic for a paper, let's say, there are some books you might need to refer to on multiple occasions. Rather than go back to the library each time, you of course check out the relevant books and take them home to your desk, where you can access them more easily.

In computing, this idea of a "memory hierarchy" remained just a theory until the development in 1962 of a supercomputer in Manchester, England, called Atlas. Its principal memory consisted of a large drum that could be rotated to read and write information, not unlike a wax phonograph cylinder. But Atlas also had a smaller, faster "working" memory built from polarized magnets. Data could be read from the drum to the magnets, manipulated there with ease, and the results then written back to the drum.

Shortly after the development of Atlas, Cambridge mathematician Maurice Wilkes realized that this smaller and faster memory wasn't just a convenient place to work with data before saving it off again. It could also be used to deliberately hold on to pieces of information likely to be needed later, *anticipating* similar future requests—and dramatically speeding up the operation of the machine. If what you needed was still in

the working memory, you wouldn't have to load it from the drum at all. As Wilkes put it, the smaller memory "automatically accumulates to itself words that come from a slower main memory, and keeps them available for subsequent use without it being necessary for the penalty of main memory access to be incurred again."

The key, of course, would be managing that small, fast, precious memory so it had what you were looking for as often as possible. To continue the library analogy, if you're able to make just one trip to the stacks to get all the books you need, and then spend the rest of the week working at home, that's almost as good as if every book in the library had already been available at your desk. The more trips back to the library you make, the slower things go, and the less your desk is really doing for you.

Wilkes's proposal was implemented in the IBM 360/85 supercomputer later in the 1960s, where it acquired the name of the "cache." Since then, caches have appeared everywhere in computer science. The idea of keeping around pieces of information that you refer to frequently is so powerful that it is used in every aspect of computation. Processors have caches. Hard drives have caches. Operating systems have caches. Web browsers have caches. And the servers that deliver content to those browsers also have caches, making it possible to *instantly* show you the same video of a cat riding a vacuum cleaner that millions of…But we're getting ahead of ourselves a bit.

The story of the computer over the past fifty-plus years has been painted as one of exponential growth year after year—referencing, in part, the famously accurate "Moore's Law" prediction, made by Intel's Gordon Moore in 1975, that the number of transistors in CPUs would double every two years. What hasn't improved at that rate is the performance of memory, which means that relative to processing time, the cost of accessing memory is also increasing exponentially. The faster you can write your papers, for instance, the greater the loss of productivity from each trip to the library. Likewise, a factory that doubles its manufacturing speed each year—but has the same number of parts shipped to it from overseas at the same sluggish pace—will mean little more than a factory that's twice as idle. For a while it seemed that Moore's Law was yielding little except processors that twiddled their thumbs ever faster and ever more of the time. In the 1990s this began to be known as the "memory

wall."

Computer science's best defense against hitting that wall has been an ever more elaborate hierarchy: caches for caches for caches, all the way down. Modern consumer laptops, tablets, and smartphones have on the order of a six-layer memory hierarchy, and managing memory smartly has never been as important to computer science as it is today.

So let's start with the first question that comes to mind about caches (or closets, for that matter). What do we do when they get full?

Eviction and Clairvoyance

> *Depend upon it there comes a time when for every addition of knowledge you forget something that you knew before. It is of the highest importance, therefore, not to have useless facts elbowing out the useful ones.*
>
> —SHERLOCK HOLMES

When a cache fills up, you are obviously going to need to make room if you want to store anything else, and in computer science this making of room is called "cache replacement" or "cache eviction." As Wilkes wrote, "Since the [cache] can only be a fraction of the size of the main memory, words cannot be preserved in it indefinitely, and there must be wired into the system an algorithm by which they are progressively overwritten." These algorithms are known as "replacement policies" or "eviction policies," or simply as caching algorithms.

IBM, as we've seen, played an early role in the deployment of caching systems in the 1960s. Unsurprisingly, it was also the home of seminal early research on caching algorithms—none, perhaps, as important as that of László "Les" Bélády. Bélády was born in 1928 in Hungary, where he studied as a mechanical engineer before fleeing to Germany during the 1956 Hungarian Revolution with nothing but a satchel containing "one change of underwear and my graduation paper." From Germany he went to France, and in 1961 immigrated to the United States, bringing his wife, "an infant son and $1,000 in my pocket, and that's it." It seems he had acquired a finely tuned sense of what to keep and what to leave behind by the time he found himself at IBM, working on cache eviction.

Bélády's 1966 paper on caching algorithms would become the most

cited piece of computer science research for fifteen years. As it explains, the goal of cache management is to minimize the number of times you can't find what you're looking for in the cache and must go to the slower main memory to find it; these are known as "page faults" or "cache misses." The optimal cache eviction policy—essentially by definition, Bélády wrote—is, when the cache is full, to evict whichever item we'll need again *the longest from now*.

Of course, knowing exactly when you'll need something again is easier said than done.

The hypothetical all-knowing, prescient algorithm that would look ahead and execute the optimal policy is known today in tribute as **Bélády's Algorithm**. Bélády's Algorithm is an instance of what computer scientists call a "clairvoyant" algorithm: one informed by data from the future. It's not necessarily as crazy as it sounds—there are cases where a system might know what to expect—but in general clairvoyance is hard to come by, and software engineers joke about encountering "implementation difficulties" when they try to deploy Bélády's Algorithm in practice. So the challenge is to find an algorithm that comes as close to clairvoyance as we can get, for all those times when we're stuck firmly in the present and can only guess at what lies ahead.

We could just try **Random Eviction**, adding new data to the cache and overwriting old data at random. One of the startling early results in caching theory is that, while far from perfect, this approach is not half bad. As it happens, just having a cache at all makes a system more efficient, regardless of how you maintain it. Items you use often will end up back in the cache soon anyway. Another simple strategy is **First-In, First-Out** (**FIFO**), where you evict or overwrite whatever has been sitting in the cache the longest (as in Martha Stewart's question "How long have I had it?"). A third approach is **Least Recently Used** (**LRU**): evicting the item that's gone the longest untouched (Stewart's "When was the last time I wore it or used it?").

It turns out that not only do these two mantras of Stewart's suggest very different policies, one of her suggestions clearly outperforms the other. Bélády compared Random Eviction, FIFO, and variants of LRU in a number of scenarios and found that LRU consistently performed the closest to clairvoyance. The LRU principle is effective because of

something computer scientists call “temporal locality”: if a program has called for a particular piece of information once, it’s likely to do so again in the near future. Temporal locality results in part from the way computers solve problems (for example, executing a loop that makes a rapid series of related reads and writes), but it emerges in the way people solve problems, too. If you are working on your computer, you might be switching among your email, a web browser, and a word processor. The fact that you accessed one of these recently is a clue that you’re likely to do so again, and, all things being equal, the program that you haven’t been using for the longest time is also probably the one that *won’t* be used for some time to come.

In fact, this principle is even implicit in the interface that computers show to their users. The windows on your computer screen have what’s called a “Z-order,” a simulated depth that determines which programs are overlaid on top of which. The least recently used end up at the bottom. As former creative lead for Firefox, Aza Raskin, puts it, “Much of your time using a modern browser (computer) is spent in the digital equivalent of shuffling papers.” This “shuffling” is also mirrored exactly in the Windows and Mac OS task switching interfaces: when you press Alt + Tab or Command + Tab, you see your applications listed in order from the most recently to the least recently used.

The literature on eviction policies goes about as deep as one can imagine—including algorithms that account for frequency as well as recency of use, algorithms that track the time of the *next*-to-last access rather than the last one, and so on. But despite an abundance of innovative caching schemes, some of which can beat LRU under the right conditions, LRU itself—and minor tweaks thereof—is the overwhelming favorite of computer scientists, and is used in a wide variety of deployed applications at a variety of scales. LRU teaches us that the next thing we can expect to need is the last one we needed, while the thing we’ll need after that is probably the second-most-recent one. And the last thing we can expect to need is the one we’ve already gone longest without.

Unless we have good reason to think otherwise, it seems that our best guide to the future is a mirror image of the past. The nearest thing to clairvoyance is to assume that history repeats itself—backward.